

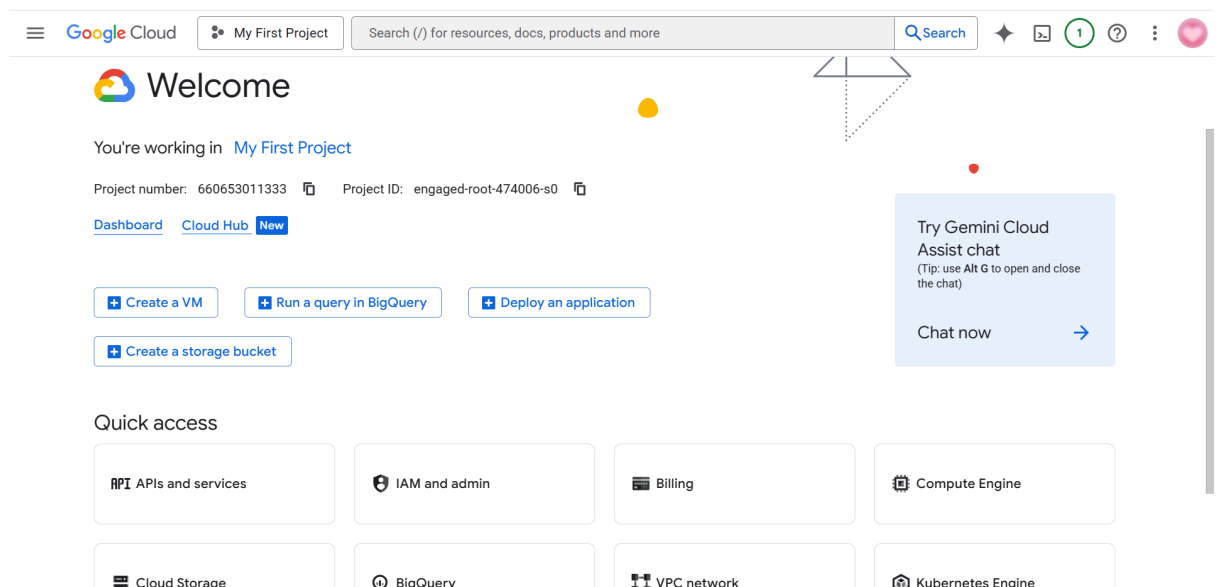
Model Context Protocol Documentation

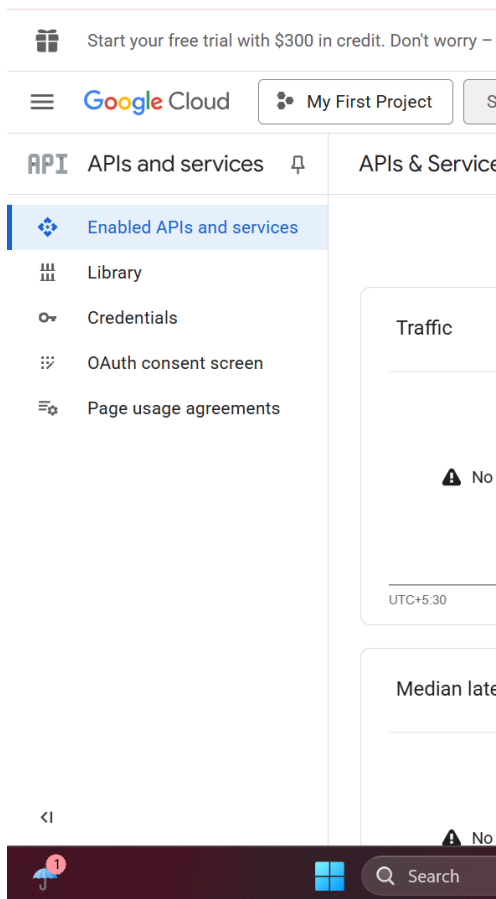
Kindly install Claude Desktop before proceeding

Ensure that after adding each server you close Claude Desktop from your Task Manager.
The servers will not work otherwise.

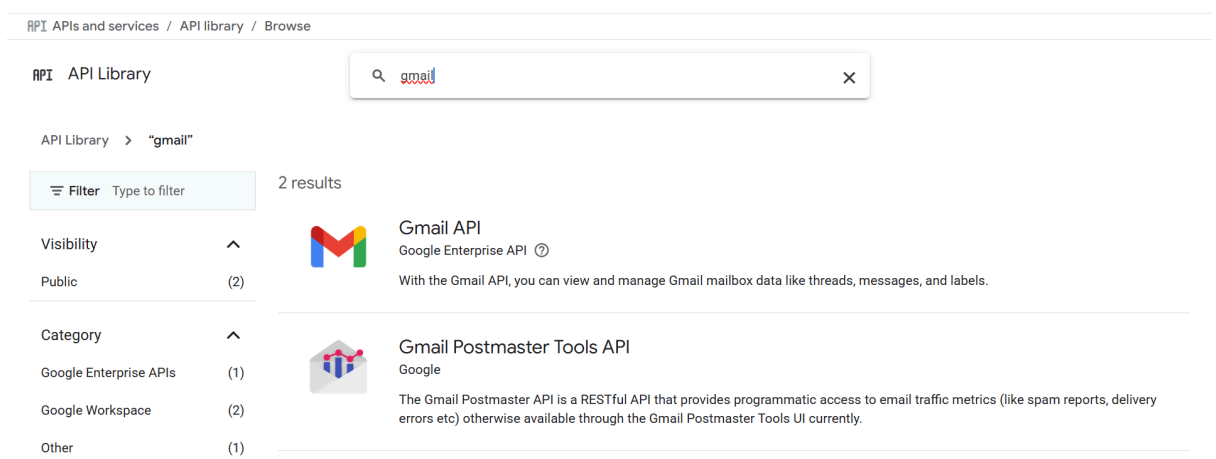
Gmail

1. Download node from this website <https://nodejs.org/en/download>
2. Go to the google cloud console to create a new project or use an existing one
3. Go to APIs and Services and go to Library

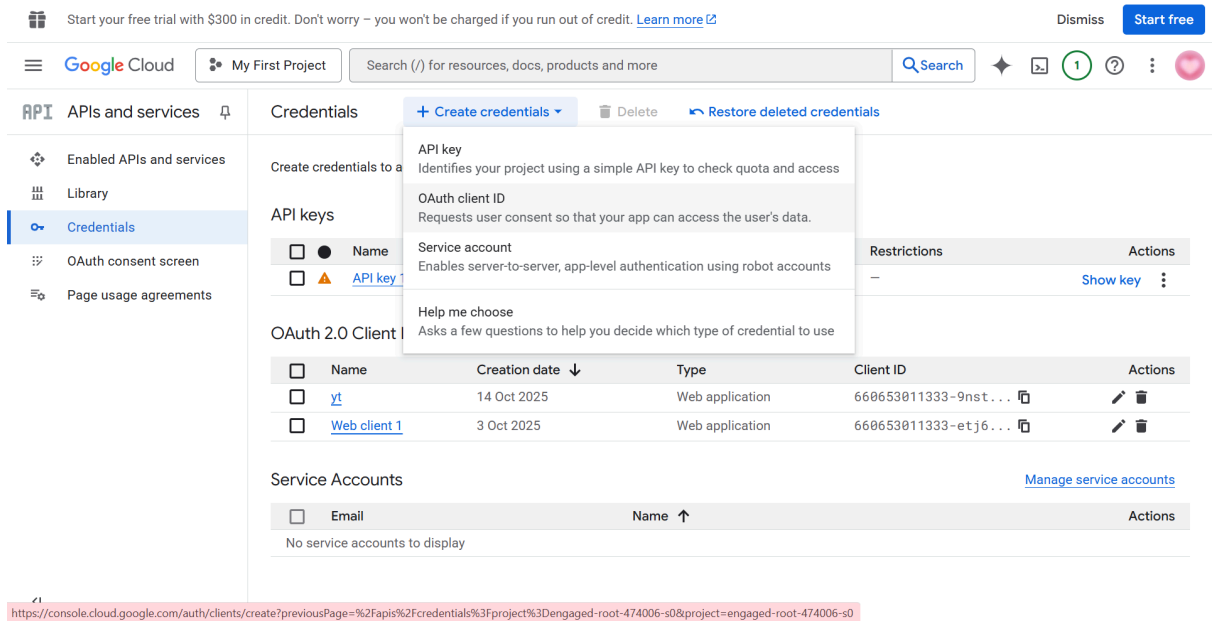




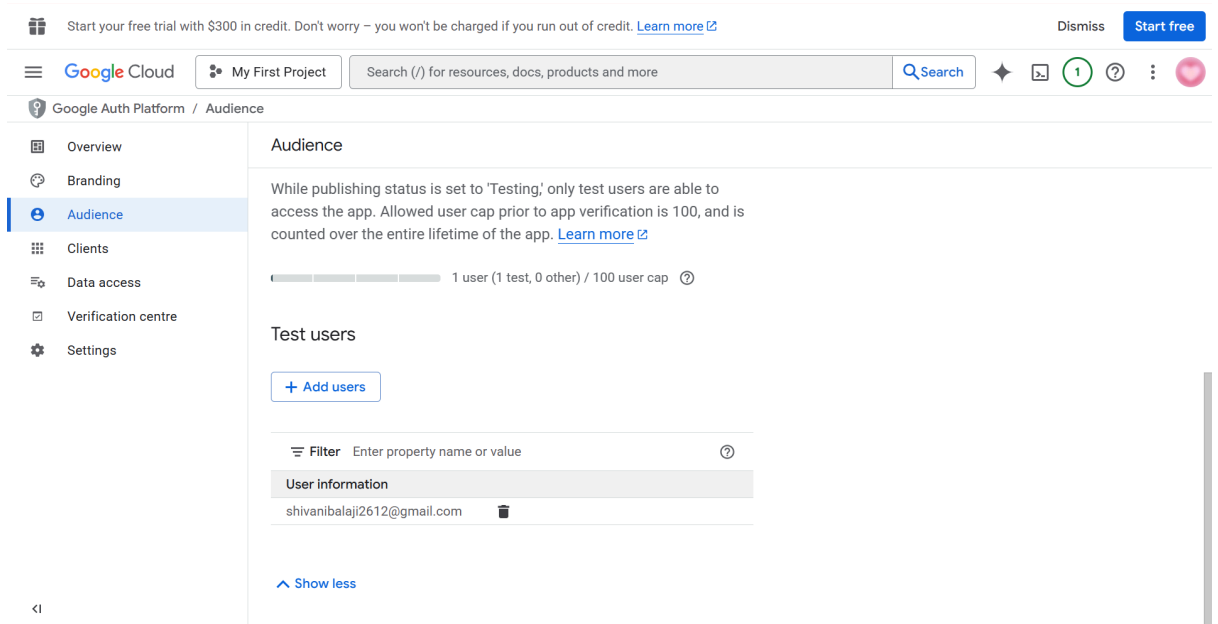
4. Enable the gmail API



5. Go back to the APIs and Services page, go to credentials and Create and create new OAuth client



6. When creating a new client ID, ensure that under Authorize redirect URIs, you include <https://localhost:4000/code>
7. After creating the client ID, ensure to copy your client ID and client secret
8. If this is your first time creating an OAuth Client, make sure to set up your OAuth Consent screen



9. Add your information in test users and under branding add the name of your project and developer information.
10. After completing the above steps, clone and install the MCP server, run the following commands

```
git clone
https://github.com/epaproditus/google-workspace-mcp-server.git
```

```
cd google-workspace-mcp-server
```

```
npm install
```

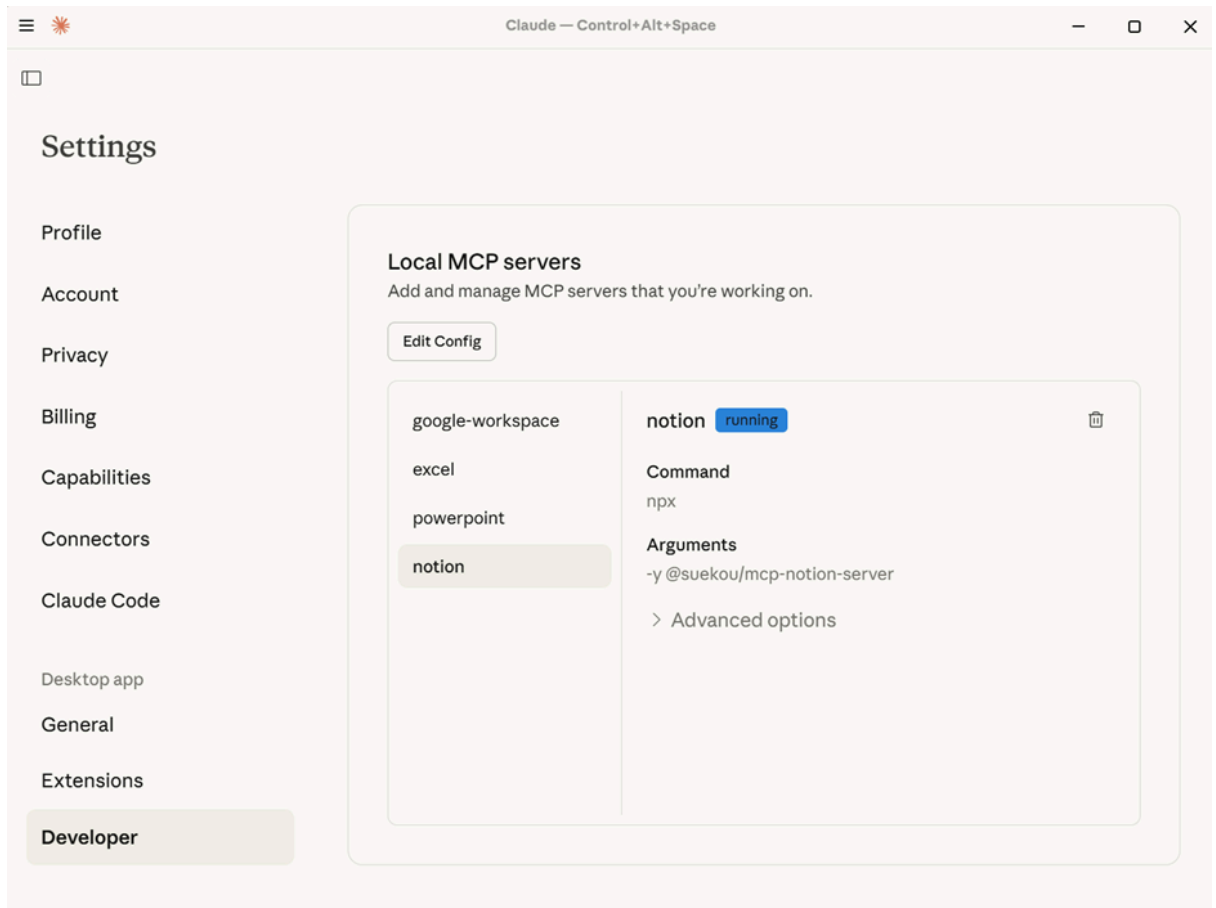
11. In the root directory of the github repo you cloned, create a file named `credentials.json` with the below content

```
{ "web": {  
  "client_id": "YOUR_CLIENT_ID",  
  "client_secret": "YOUR_CLIENT_SECRET",  
  "redirect_uris": ["http://localhost:4100/code"],  
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",  
  "token_uri": "https://oauth2.googleapis.com/token"  
}}
```

12. Under `your_client_id` and `your_client_secret`, paste the actual client id and secret you copied when creating your OAuth Client
13. Obtain refresh token by running the below command inside the repo

```
node get-refresh-token.js
```

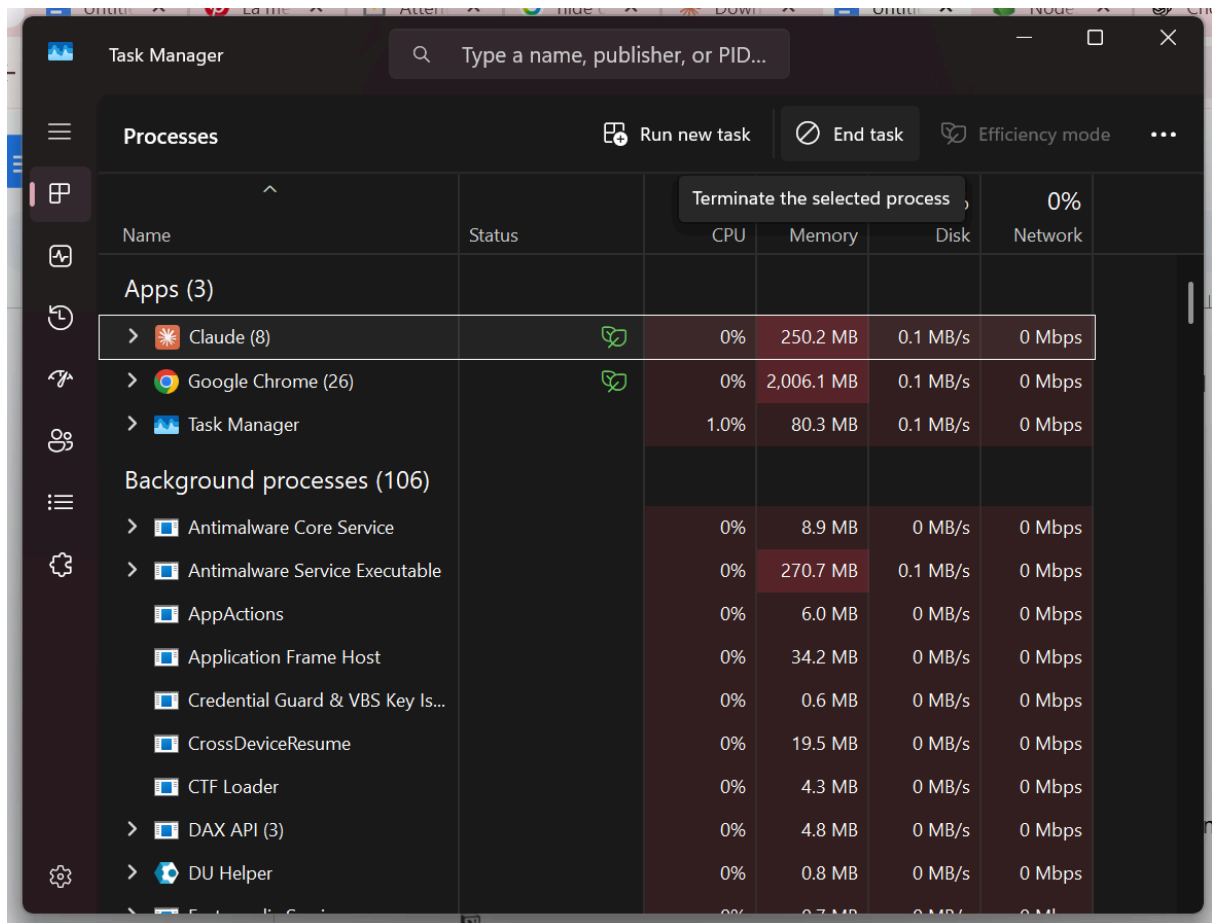
14. Approve the Google OAuth Authentication screen that pops up in the browser, you can only access this screen if you have added your email address under test users as mentioned in step 9. Approve them and save the credentials to `token.json`
15. Add the server to the claude config file, you can access this file by opening claude desktop -> file -> settings
16. Under settings scroll down to developer and click on edit config



17. In the config file, add our google-workspace server as follows

```
{ "mcpServers": {  
  "google-workspace": {  
    "command": "node",  
    "args": [  
      '/path/to/google-workspace-server/build/index.js'  
    ],  
    "env": {  
      "GOOGLE_CLIENT_ID": "your_client_id",  
      "GOOGLE_CLIENT_SECRET": "your_client_secret",  
      "GOOGLE_REFRESH_TOKEN": "your_refresh_token"  
    }  
  }  
}
```

18. Save the file, now open Task Manager by pressing Ctrl+Shift+Esc. End the claude task and reopen it to access the google-workspace MCP server



19. Ensure you perform step 18 after every new server added.

Excel

1. In the `claude_config_file`, add this inside `mcpServers`

```
"excel": {
  "command": "cmd",
  "args": [
    "/c",
    "npx",
    "--yes",
    "@negokaz/excel-mcp-server"
  ],
  "env": {
```

```
"EXCEL_MCP_PAGING_CELLS_LIMIT": "4000"
```

```
}
```

```
},
```

2. Perform step 18 under the gmail steps, you can now access the excel server.

Powerpoint presentation

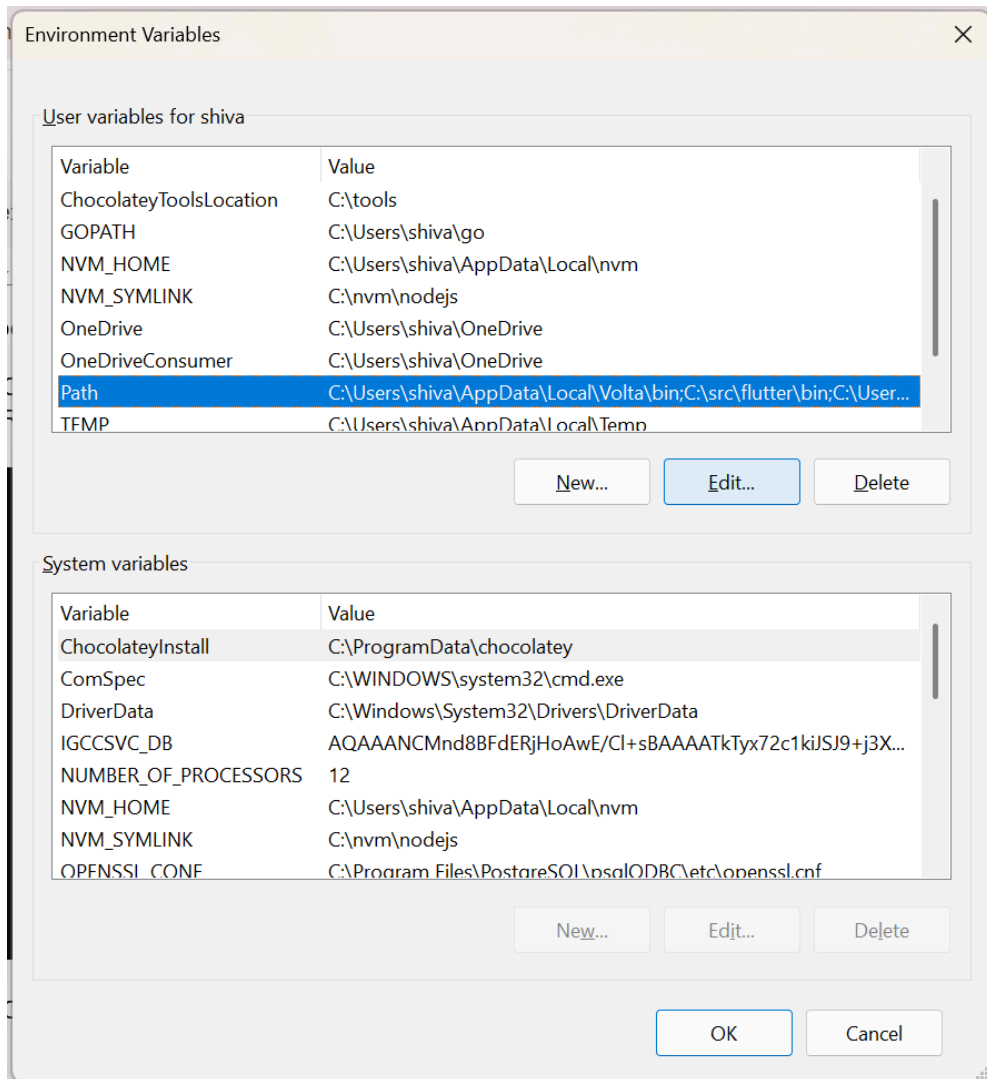
1. Open powershell as administrator and run pip install uv
2. Run pip show uv to get the location of the executable file of the uv packet manager.

```
PS C:\Users\shiva> pip install uv
Requirement already satisfied: uv in c:\users\shiva\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (0.9.2)

[notice] A new release of pip is available: 24.0 -> 25.3
[notice] To update, run: C:\Users\shiva\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
PS C:\Users\shiva> where uv
PS C:\Users\shiva> get-location uv
Get-Location : A positional parameter cannot be found that accepts argument 'uv'.
At line:1 char:1
+ get-location uv
+ ~~~~~
+ CategoryInfo          : InvalidArgument: (:) [Get-Location], ParameterBindingException
+ FullyQualifiedErrorId : PositionalParameterNotFound,Microsoft.PowerShell.Commands.GetLocationCommand

PS C:\Users\shiva> where uv
PS C:\Users\shiva> pip show uv
Name: uv
Version: 0.9.2
Summary: An extremely fast Python package and project manager, written in Rust.
Home-page: https://pypi.org/project/uv/
Author:
Author-email: "Astral Software Inc." <hey@astral.sh>
License:
Location: C:\Users\shiva\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages
Requires:
Required-by:
PS C:\Users\shiva> |
```

3. Open start and search edit environment variables. Click on it, under user variables -> path, click on edit and add this location to it



4. Clone the git repo via the command

```
git clone https://github.com/Ichigo3766/powerpoint-mcp.git
```

5. Open the `claude_desktop_config.json` file, add the following to the `mcpServers`.

```
"powerpoint": {  
  "command": "uv",  
  "args": [  
    "--directory",  
    PATH_TO_POWERPOINT_MCP_GIT,  
    "run",  
    "powerpoint",
```



```

    "--folder-path",

    PATH_TO_WHERE_PPTS_ARE_TO_BE_SAVED

]

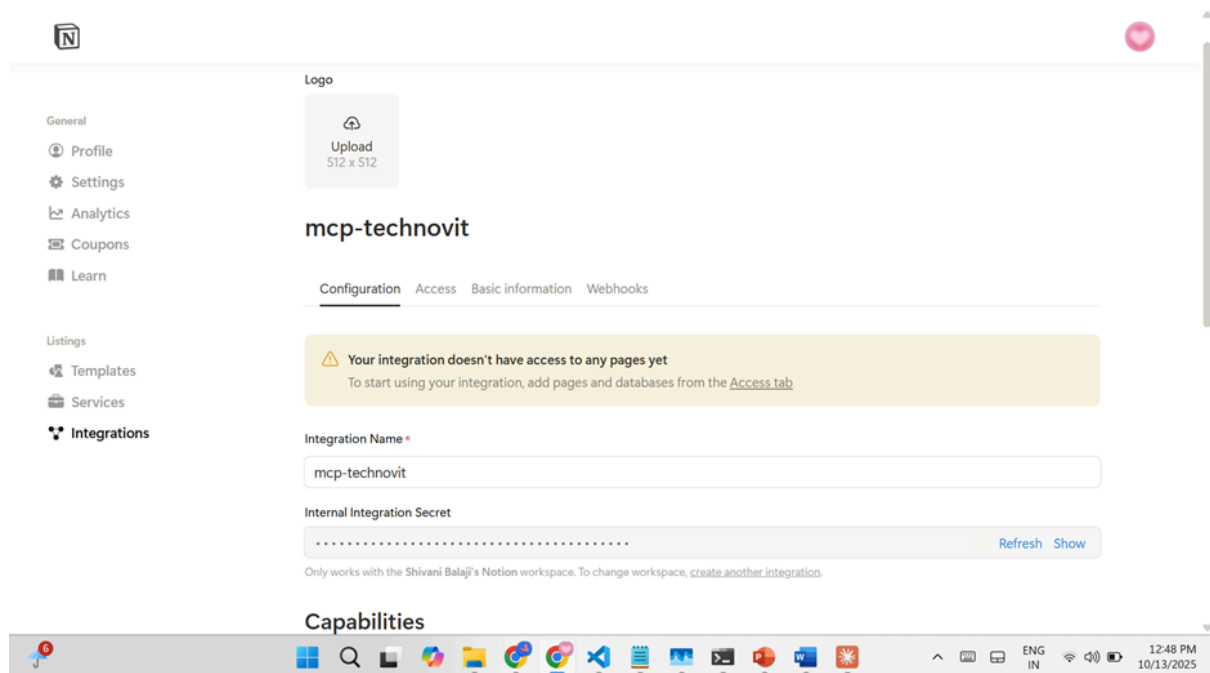
},

```

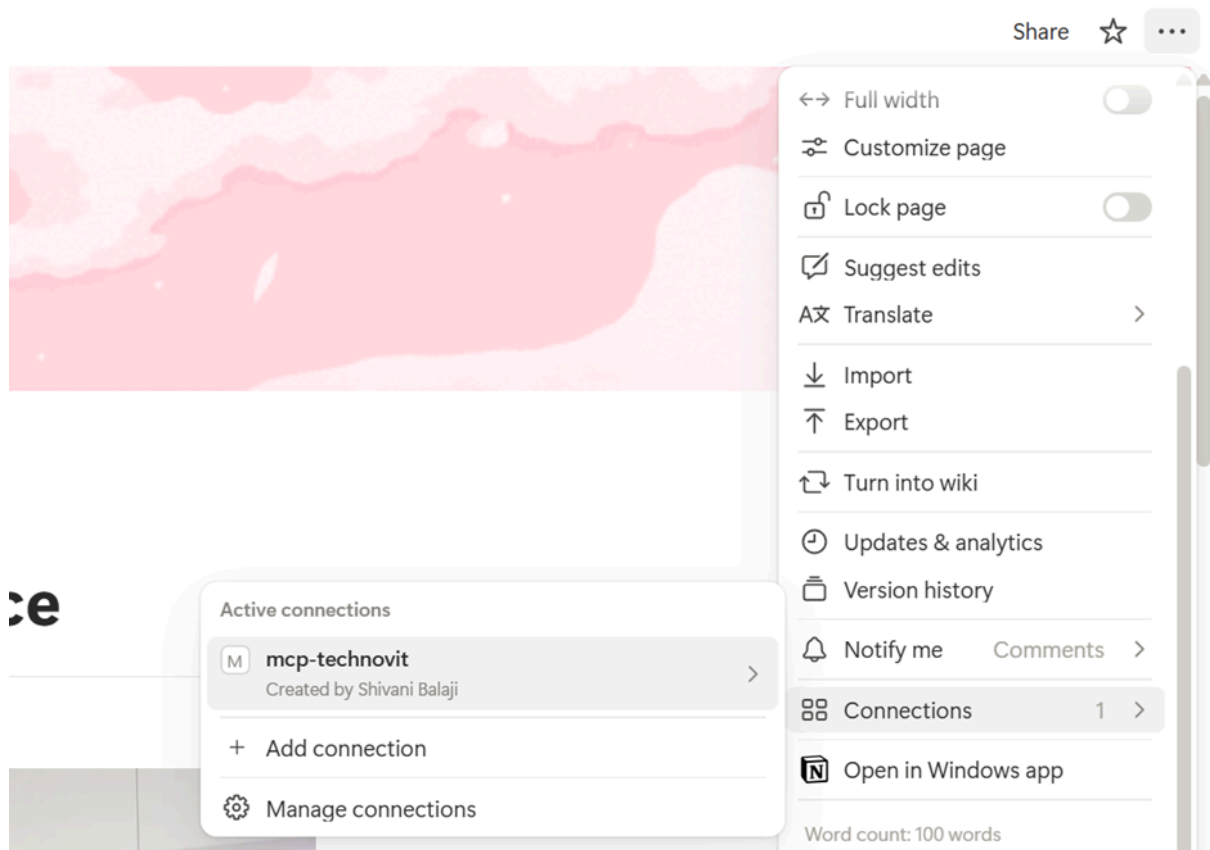
6. Close Claude via the task manager and reopen it, you should now be able to generate simple powerpoint presentations via Claude.

Notion

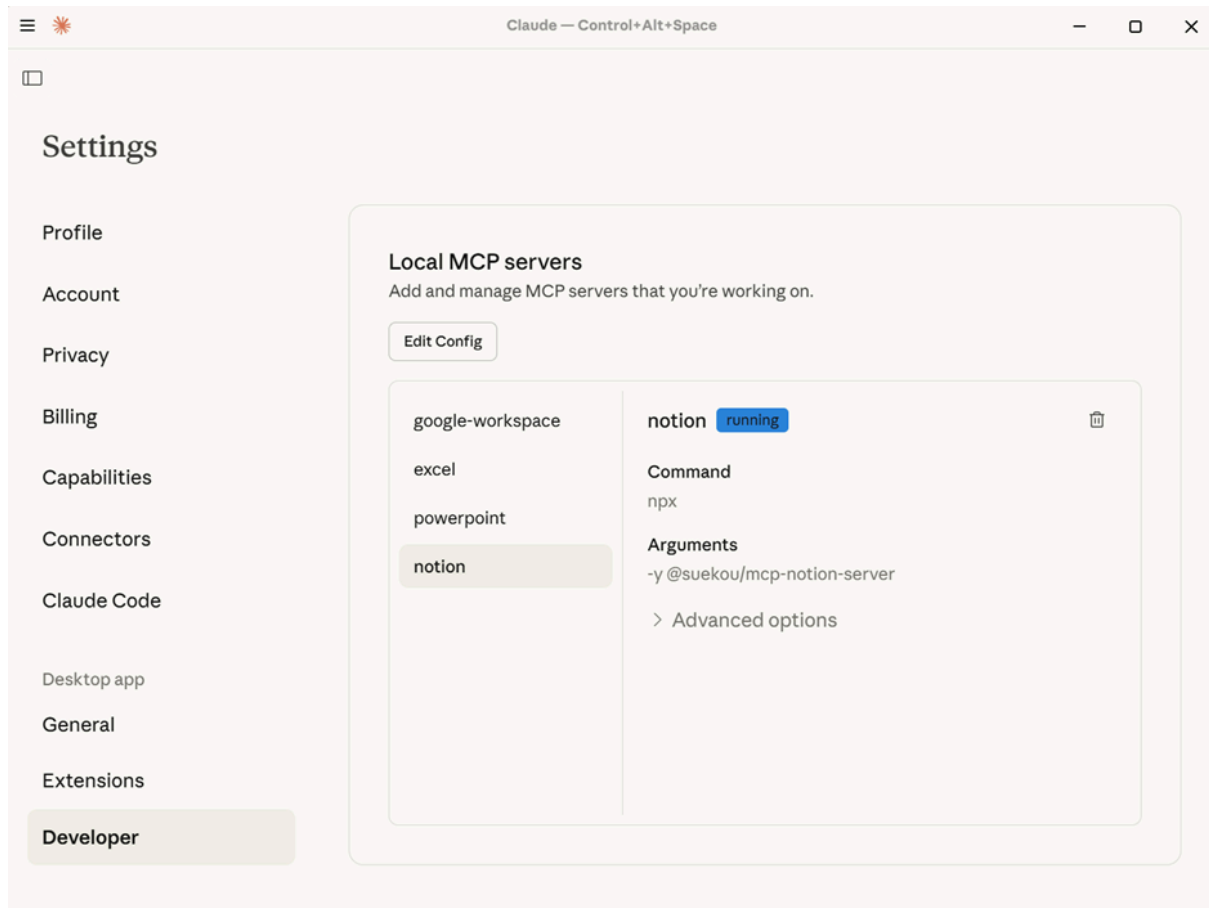
1. Go to the url <https://www.notion.so/profile/integrations>
2. Create a new integration and copy the internal integration secret as shown in the screenshot below



3. Open the page / database that you would like to use with claude in notion. In the top-right corner click on the 3 dots and scroll down. Click on connections and search up the integration we just created.



4. Open claude, in the top-left corner there is a menu. Under file, open settings.
5. Inside settings, scroll down to the developer section to open the `claude_desktop_config.json` file



6. After opening the config file, inside mcpServers add the following

```
{
  "mcpServers": {
    "notion": {
      "command": "npx",
      "args": ["-y", "@suekou/mcp-notion-server"],
      "env": {
        "NOTION_API_TOKEN": "internal_integration_secret"
      }
    }
  }
}
```

7. Close Claude from the task manager, verify that the server is running in the developer section in settings.
8. Now, we can request Claude to handle our notion pages in plain English.

Youtube

1. Carry out the following commands

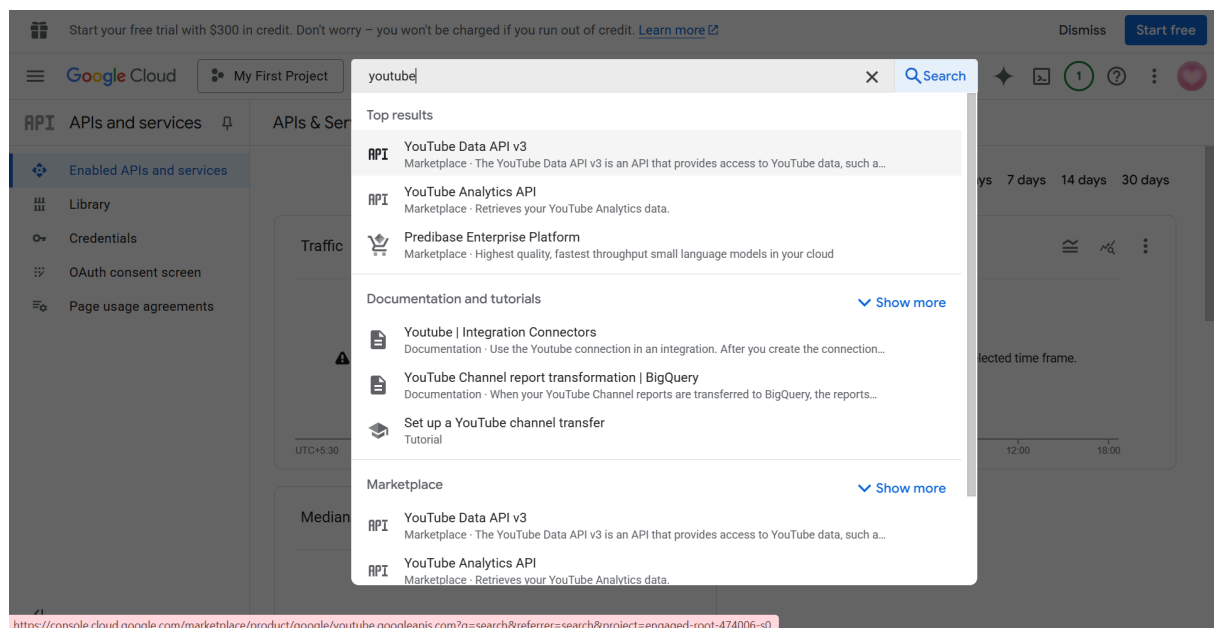
```
git clone  
https://github.com/icraft2170/youtube-data-mcp-server.git
```

```
cd youtube-data-mcp-server
```

```
npm install
```

```
npm run build
```

2. The npm run build command generates a dist folder with an [index.js](#) file inside it.
3. In order to get an API key for youtube, go to google cloud console.
4. In the navigation menu, go to APIs and Libraries->Enabled APIs and Services, look for Youtube in the search bar.



5. Enable this API and click on Manage. Under credentials click on Create Credentials -> Create API Key

← API/Service details ■ Disable API



YouTube Data API v3

The YouTube Data API v3 is an API that provides access to YouTube data, such as videos, playlists, and channels.

By Google

Service name	Type	Status
youtube.googleapis.com	Public API	Enabled

API key
Identifies your project using a simple API key to check quota and access

OAuth client ID
Requests user consent so that your app can access the user's data.

Help me choose
Asks a few questions to help you decide which type of credential to use

Metrics Quotas and system limits Credentials

+ [Create credentials](#) 🗑 Delete

Credentials compatible with this API

To view all credentials visit [Credentials in APIs and services](#)

API keys

- Copy the API key after creating it.
- Ensure you have node installed on your system.
- Open the `claude_desktop_config.json` file and add the following to `mcpServers`

```
"youtube": {
  "command": "node",
  "args": [
    PATH_TO_INDEX.JS
  ],
  "env": {
    "YOUTUBE_API_KEY": API_KEY,
    "YOUTUBE_TRANSCRIPT_LANG": "en"
  }
}
```

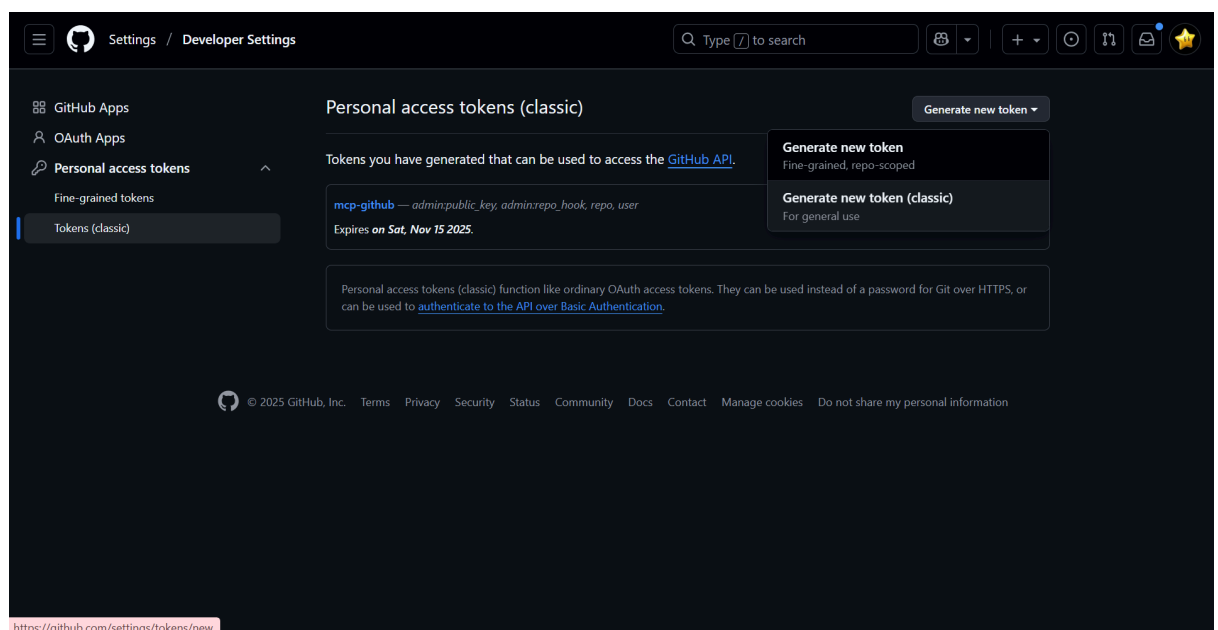
SQLite

1. Ensure you have SQLite on your system and have created at least one pre-populated SQLite database object.
2. Also ensure you have packet manager uv installed as mentioned in the powerpoint section.
3. Add the following to your claude_desktop_config.json file

```
"sqlite": {  
  
  "command": "uv",  
  
  "args": [  
  
    "run",  
  
    "mcp-server-sqlite",  
  
    "--db-path",  
  
    PATH_TO_SQLITE_DB  
  
  ]  
  
},
```

Github

1. Ensure you have a Github account. Navigate to this URL to create a Personal Access Token (PAT).



2. Generate a new classic token, and give it the above privileges (as mentioned near mcp-github). Copy the access token.

3. Go to this URL <https://github.com/github/github-mcp-server/releases> to download the release binary for the claude github mcp for your system.
4. After downloading the release binary, unzip the folder and save it to your desired location.
5. Copy the path in which the .exe / Application file lives. Add it to user variables -> path as mentioned in the powerpoint section.
6. Add the following to your claude_desktop_config.json.

```
"github": {  
  
  "command": "github-mcp-server",  
  
  "args": [  
  
    "stdio"  
  
  ],  
  
  "env": {"GITHUB_PERSONAL_ACCESS_TOKEN":  
PERSONAL_ACCESS_TOKEN}  
  
}
```